



طراحی و پیاده سازی پردازنده تک چرخه ۳۲ بیتی

معماری کامپیوتر – گزارش کار تمرین عملی پنجم

استاد

دکتر سربازی آزاد

دانشگاه صنعتی شریف

بهار ۱۴۰۵

محمد مهدی مرادی – ۴۰۳۱۰۶۶۸۱

۱-مقدمه.....	۳
۲- مشخصات ورودی‌ها و خروجی‌های پردازنده.....	۳
۳-معماری کلی و مسیر داده (Datapath).....	۳
۴-تشریح ساختار و پیاده‌سازی سخت‌افزاری ماژول‌ها.....	۴
۵-نحوه مدیریت محدودیت‌های حافظه (Word Alignment).....	۱۰
۶- تحلیل نتایج شبیه‌سازی و بررسی وارنینگ‌ها.....	۱۰
۷-نتیجه گیری.....	۱۲

۱. مقدمه:

هدف از این تمرین عملی، طراحی، ساختاردهی و پیاده‌سازی یک هسته پردازنده ۳۲ بیتی تک‌چرخه (Single-Cycle) کامل با قابلیت اجرای مجموعه‌ای از دستورات محاسباتی، منطقی، انتقال داده و پرش بر پایه معماری MIPS است. این پردازنده به زبان Verilog پیاده‌سازی شده و قادر است برنامه‌های پیچیده‌ای مانند محاسبه دنباله فیبوناچی، ضرب محاسباتی و تبدیل مبنای اعداد به مبنای ۳ را به صورت سخت‌افزاری اجرا کند.

۲. مشخصات ورودی‌ها و خروجی‌های پردازنده:

پردازنده طراحی شده دارای خطوط ارتباطی استاندارد زیر با حافظه دستورات (Instruction Memory) و حافظه داده (Data Memory) است:

الف) سیگنال‌های ورودی:

- **Clk**: سیگنال ساعت سراسری سیستم برای سنکرون‌سازی ثبات‌ها.
- **Rst**: سیگنال بازنشانی (Reset) برای مقداردهی اولیه به PC و وضعیت پردازنده.
- **InstrIn [۳۱:۰]**: دستور ۳۲ بیتی خوانده شده از حافظه دستورات.
- **DataIn [۳۱:۰]**: داده ۳۲ بیتی خوانده شده از حافظه داده (در دستورات لود).

ب) سیگنال‌های خروجی:

- **InstrAddr [۳۱:۰]**: نشانی دستور فعلی (ارسال به حافظه دستورات به صورت کلمه‌ای).
- **DataAddr [۳۱:۰]**: نشانی داده برای دسترسی به حافظه داده (ارسال به صورت کلمه‌ای).
- **DataWrite**: سیگنال فعال‌ساز نوشتن در حافظه داده (MemWrite).
- **DataOut [۳۱:۰]**: داده‌ای که باید در حافظه داده ذخیره شود (در دستورات استور).
- **R۰ تا R۳۱**: خروجی ۳۲ ثبات موجود در بانک ثبات برای مانیتورینگ و ارزیابی در تست‌بنچ.

۳. معماری کلی و مسیر داده (Datapath):

این پردازنده به صورت تک‌چرخه (Single-Cycle) طراحی شده است؛ به این معنی که تمام مراحل اجرای یک دستور (فراخوانی دستور، رمزگشایی، فراخوانی ثبات‌ها، اجرای محاسبات ALU، دسترسی به حافظه داده و بازنویسی در بانک ثبات) همگی در طول یک چرخه ساعت انجام می‌شوند. فرکانس کاری مدار و طول چرخه ساعت بر اساس طولانی‌ترین مسیر بحرانی (Critical Path) که مربوط به دستورات دسترسی به حافظه (مانند LW) است، تعیین می‌شود.

۴. تشریح ساختار و پیاده‌سازی سخت‌افزاری ماژول‌ها:

۴-۱. ماژول شمارنده برنامه (PC):

ثبات PC یک ثبات ۳۲ بیتی ترتیبی است که در لبه بالارونده کلاک، آدرس دستور بعدی (nextPC) را ثبت می‌کند. در صورت فعال بودن سیگنال Rst، مقدار این ثبات برابر صفر قرار می‌گیرد. برای تضمین پایداری در شبیه‌سازی، مقدار اولیه آن در بلوک initial نیز صفر شده است.

```
module PC (  
    input clk,  
    input rst,  
    input [31:0] nextPC,  
    output reg [31:0] pc  
);  
    initial begin  
        pc = 32'b0;  
    end  
  
    always @(posedge clk or posedge rst) begin  
        if (rst) pc <= 32'b0;  
        else pc <= nextPC;  
    end  
endmodule
```

۲-۴. ماژول بانک ثبت (Register File):

بانک ثبت شامل ۳۲ ثبت ۳۲ بیتی است. این ماژول دارای دو درگاه خواندن همزمان ترکیبی و یک درگاه نوشتن سنکرون با لبه کلاک است.

• **محدودیت ثبت صفر (R۰):** طبق صورت پروژه، ثبت R۰ همواره باید مقدار صفر داشته باشد. برای این کار، مدار ورودی نوشتن مجهز به یک ساختار منطقی شده است که اگر آدرس نوشتن برابر صفر باشد ($\text{writeReg} \neq 5'b0$)، عملیات بازنویسی نادیده گرفته می‌شود.

• **مقداردهی اولیه:** جهت جلوگیری از به وجود آمدن مقادیر ناشناخته (X) در ابتدای شبیه‌سازی، تمام ثبت‌ها در یک حلقه initial برابر صفر منطقی قرار گرفته‌اند.

```
module RegisterFile (
    input clk,
    input regWrite,
    input [4:0] readReg1, readReg2, writeReg,
    input [31:0] writeData,
    output [31:0] readData1, readData2,
    output [31:0] R0, R1, R2, R3, R4, R5, R6, R7, R8, R9, R10,
    output [31:0] R11, R12, R13, R14, R15, R16, R17, R18, R19, R20,
    output [31:0] R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31
);
    reg [31:0] registers [31:0];
    integer i;

    initial begin
        for (i = 0; i < 32; i = i + 1) begin
            registers[i] = 32'b0;
        end
    end

    assign readData1 = registers[readReg1];
    assign readData2 = registers[readReg2];

    assign {R31, R30, R29, R28, R27, R26, R25, R24, R23, R22, R21, R20, R19, R18, R17, R16,
            R15, R14, R13, R12, R11, R10, R9, R8, R7, R6, R5, R4, R3, R2, R1, R0} =
        {registers[31], registers[30], registers[29], registers[28], registers[27],
         registers[26], registers[25], registers[24], registers[23], registers[22],
         registers[21], registers[20], registers[19], registers[18], registers[17],
         registers[16], registers[15], registers[14], registers[13], registers[12],
         registers[11], registers[10], registers[9], registers[8], registers[7],
         registers[6], registers[5], registers[4], registers[3], registers[2],
         registers[1], registers[0]};

    always @(posedge clk) begin
        if (regWrite && (writeReg != 5'b0)) begin
            registers[writeReg] <= writeData;
        end
    end
endmodule
```

۳-۴. ماژول‌های محاسباتی ضرب‌کننده (Multiplier) و مقسوم‌علیه (Divider):

یکی از مهم‌ترین محدودیت‌های این پروژه، ممنوعیت استفاده از عملگرهای ترکیبی خطی * و / و در عین حال لزوم اجرای تک‌چرخه‌ای مدار است. ماژول‌های ترتیبی مبتنی بر کلاک به دلیل نیاز به چندین ساینکل کلاک، ساختار تک‌چرخه را نقض می‌کردند. بنابراین از ساختارهای کاملاً ترکیبی (Combinational Loops) با حلقه‌های تفکیک‌شده استفاده شد:

• **ضرب‌کننده:** با استفاده از الگوریتم Shift and Add در یک بلوک always @(*) پیاده‌سازی شده است که بیت‌های عملوند دوم را بررسی کرده و جابه‌جایی محاسباتی را اعمال می‌کند.

• **مقسوم‌علیه:** از الگوریتم تقسیم Shift and Subtract استفاده می‌کند که در یک ساینکل کلاک، خارج‌قسمت و باقی‌مانده ۳۲ بیتی را به صورت ترکیبی استخراج می‌نماید.

```
module Multiplier (
    input [31:0] a,
    input [31:0] b,
    output reg [31:0] result
);
    integer i;
    always @(*) begin
        result = 32'b0;
        for (i = 0; i < 32; i = i + 1) begin
            if (b[i]) result = result + (a << i);
        end
    end
endmodule

module Divider (
    input [31:0] a,
    input [31:0] b,
    output reg [31:0] result
);
    integer i;
    reg [31:0] quotient, remainder;
    always @(*) begin
        quotient = 32'b0;
        remainder = 32'b0;
        for (i = 31; i >= 0; i = i - 1) begin
            remainder = (remainder << 1) | ((a >> i) & 1'b1);
            if (remainder >= b && b != 0) begin
                remainder = remainder - b;
                quotient[i] = 1'b1;
            end
        end
        result = quotient;
    end
endmodule
```

۴-۴. واحد حساب و منطق (ALU):

این واحد عملوندهای ورودی را دریافت کرده و بر اساس سیگنال کنترلی aluControl خروجی را محاسبه می‌کند. این ماژول نمونه‌هایی از ضرب‌کننده و مقسوم‌علیه ترکیبی را در خود دارد. وظایف این واحد شامل عملیات جمع (ADD/ADDI/LW/SW)، تفریق (SUB/SUBI/BEQ)، ضرب (MUL)، تقسیم (DIV) و شیفت به چپ برای لود مقدار ثابت بالا (LUI) است. خروجی Zero نیز جهت ارزیابی شرط پرش در دستور BEQ تعبیه شده است.

```
module ALU (  
    input [31:0] a,  
    input [31:0] b,  
    input [3:0] aluControl,  
    output reg [31:0] result,  
    output zero  
);  
  
wire [31:0] mult_res, div_res;  
  
Multiplier m1 (a, b, mult_res);  
Divider d1 (a, b, div_res);  
  
always @(*) begin  
    case (aluControl)  
        4'b0000: result = a + b;  
        4'b0001: result = a - b;  
        4'b0100: result = mult_res;  
        4'b0101: result = div_res;  
        4'b0110: result = b << 16;  
        default: result = 32'b0;  
    endcase  
end  
  
assign zero = (result == 32'b0);  
endmodule
```


۵-۴. واحد کنترل (Control Unit):

این واحد مدار ترکیبی است که فیلدهای Opcode و Funct دستور را دریافت کرده و سیگنال‌های کنترلی مسیر داده را تولید می‌کند. این سیگنال‌ها شامل Jal، Branch، Jump، MemWrite، MemRead، ALUSrc، RegWrite، RegDst، ALUOp و Jr، MemToReg هستند. پشتیبانی از دستورات خاص مانند LUI (با آپ‌کد ۰۰۱۱۱۱)، SUBI (با آپ‌کد ۰۰۱۰۰۱) و پرش غیرمستقیم JR (با فیلد funct برابر ۰۰۱۰۰۰) به طور کامل در جدول حالت این ماژول لحاظ شده است.

```
module ControlUnit (
    input [5:0] opcode,
    input [5:0] funct,
    output reg regDst, regWrite, aluSrc, memRead, memWrite, branch, jump, jal, jr,
    output reg [1:0] memToReg,
    output reg [3:0] aluOp
);
always @(*) begin
    regDst = 0; regWrite = 0; aluSrc = 0; memRead = 0; memWrite = 0;
    memToReg = 0; branch = 0; jump = 0; jal = 0; jr = 0; aluOp = 4'b0000;

    case (opcode)
        6'b000000: begin
            if (funct == 6'b001000) begin
                jr = 1;
            end else begin
                regDst = 1; regWrite = 1;
                case (funct)
                    6'b100000: aluOp = 4'b0000;
                    6'b100010: aluOp = 4'b0001;
                    6'b011000: aluOp = 4'b0100;
                    6'b011010: aluOp = 4'b0101;
                endcase
            end
        end
        6'b100011: begin regWrite = 1; aluSrc = 1; memRead = 1; memToReg = 1; aluOp = 4'b0000; end
        6'b101011: begin aluSrc = 1; memWrite = 1; aluOp = 4'b0000; end
        6'b001000: begin regWrite = 1; aluSrc = 1; aluOp = 4'b0000; end
        6'b001001: begin regWrite = 1; aluSrc = 1; aluOp = 4'b0001; end
        6'b001111: begin regWrite = 1; aluSrc = 1; aluOp = 4'b0110; end
        6'b000100: begin branch = 1; aluOp = 4'b0001; end
        6'b000010: jump = 1;
        6'b000011: begin jump = 1; jal = 1; regWrite = 1; memToReg = 2; end
    endcase
end
endmodule
```


۴-۶. واحد محاسبات آدرس پرش و انشعاب (AddressCalc):

این واحد وظیفه تعیین مقدار آدرس بعدی PC را بر عهده دارد:

• در حالت عادی: $PC + 4$

• در دستور **BEQ** (در صورت برقرار بودن شرط): $PC + 4 + (\text{SignExtImm} \ll 2)$

• در دستور **JAL**: ادغام ۴ بیت بالای $PC + 4$ با آدرس پرش ۲۶ بیتی شیفت یافته.

• در دستور **JR**: کپی مستقیم مقدار ثبات rs خوانده شده به عنوان آدرس بعدی.

```
module AddressCalc (
    input [31:0] pc,
    input [31:0] immExt,
    input [25:0] jumpAddr,
    input [31:0] rs_val,
    input jump,
    input branch,
    input zero,
    input jr,
    output [31:0] nextPC
);
    wire [31:0] pcPlus4 = pc + 4;

    assign nextPC = jr ? rs_val :
        jump ? {pcPlus4[31:28], jumpAddr, 2'b00} :
        (branch && zero) ? (pcPlus4 + (immExt << 2)) : pcPlus4;
endmodule
```

۵. نحوه مدیریت محدودیت‌های حافظه (Word Alignment):

میدانیم که دسترسی در حافظه به کلمات ۴ بیتی است و دو بیت سمت راست نشانی باید نادیده گرفته شوند. در سخت‌افزار این پردازنده، متغیرهای pc و aluResult بایت-آدرس (Byte-addressable) هستند و آدرس‌ها به صورت ۴، ۸، ۱۲ و... تغییر می‌کنند. اما حافظه شبیه‌سازی تست‌بنچ کلمه-آدرس (Word-addressable) است. برای تطابق کامل با این محدودیت، آدرس‌های ارسالی به خارج از پردازنده با دو بیت شیفت به راست مشخص شده‌اند: `assign InstrAddr = pc >> ۲; assign DataAddr = aluResult >> ۲`; این کار تضمین می‌کند که آدرس در فضای تست‌بنچ به صورت کلمات مرتب‌شده (ایندکس‌های ۰، ۱، ۲، ۳ و...) خوانده و نوشته شود.

۶. تحلیل نتایج شبیه‌سازی و بررسی وارنینگ‌ها:

پس از اجرای شبیه‌سازی مدار با ابزار Icarus Verilog (iverilog) و اجرای آن با ابزار vvp، خروجی سیستم با موفقیت کامل پیام ACCEPTED را چاپ کرد و تمامی ۲۰۸ مرحله ارزیابی تست‌بنچ را بدون خطای منطقی پاس نمود.

تحلیل نتایج نهایی ارزیابی:

- مقدار محاسبه‌شده برای دستور فیبوناچی ۱۰ برابر با ۵۵ (کاملاً درست).
- مقدار محاسبه‌شده برای دستور فیبوناچی ۱۱ برابر با ۸۹ (کاملاً درست).
- حاصل ضرب محاسباتی دو عدد (۵۵ * ۸۹) برابر با ۴۸۹۵ (کاملاً درست).
- ارقام استخراج شده در مبنای ۳ به صورت معکوس: ۲ ۲ ۰ ۱ ۰ ۲ ۰ ۲ که نشان‌دهنده عدد محاسباتی درست در مبنای ۳ (۲۰۲۰۱۰۲۲) است.

```

WARNING: Invalid address to read : 00000004
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000004
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000004
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000028
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000078
WARNING: Invalid address to read : 00000000
fib(10) = exp: 55  real: 55
fib(11) = exp: 89  real: 89
product = exp: 4895  real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
HW5/tb.v:324: $finish called at 424 (1s)

```

تحلیل پیام‌های اخطار (WARNING: Invalid address to read):

تست‌بنچ در حین شبیه‌سازی اخطارهایی مبنی بر معتبر نبودن آدرس‌های خواندن چاپ می‌کند. این اخطارها به دلیل فیزیک مدار تک‌چرخه و رفتار مدارات ترکیبی است. هنگامی که مقادیر در مسیر داده تغییر می‌کنند، تا زمان تثبیت نهایی سیگنال‌ها در سیم‌ها (اطلاعات گذرا یا Glitch)، خروجی ALU برای چند پیکوثانیه مقادیر نامعتبر به خود می‌گیرد. از آنجا که تست‌بنچ به تغییرات لحظه‌ای آدرس حساس است، این وارنینگ‌ها را چاپ می‌کند. اما از آنجا که نمونه‌برداری نهایی داده‌ها در لبه ساعت (پایان چرخه) انجام می‌شود و در آن لحظه سیگنال‌ها کاملاً پایدار و دقیق هستند، هیچ خطایی در اجرای برنامه رخ نداده و برنامه با موفقیت تأیید (ACCEPTED) می‌شود.

۷. نتیجه‌گیری:

در این پروژه، یک پردازنده ۳۲ بیتی تک‌چرخه منطبق بر ساختار معماری کامپیوتر پیشرفته با موفقیت طراحی و پیاده‌سازی شد. چالش‌های مهم پروژه از جمله پیاده‌سازی ضرب و تقسیم تک‌چرخه‌ای بدون استفاده از اپراتورهای پیش‌فرض سیستم، تدارک منطق پرش‌های پیچیده مانند JAL و JR، دسترسی کلمه‌ای به حافظه و حفاظت سخت‌افزاری از ثبات R_0 با موفقیت حل شدند. نتایج تست‌بنچ صحت عملکرد بخش‌های مختلف مسیر داده و واحد کنترل را به طور کامل اثبات نمود.